# BGP 101: Lab Guide

Carlos M. Martinez

2026-10-18

Version v0.2.0

Carlos M. Martinez
XT6 Labs

**Version history**

| Version | Date |
|---|---|

# Table of contents


**About this course** 5
References & Further Reading . . . 5

**1 Lab 1 - Establishing Basic BGP Connectivity** 7
1.1 Objectives . . . 7
1.2 Working with the Lab Environment . . . 7
1.3 Topology . . . 8
1.4 Deployment . . . 8
1.5 Step 1: Connectivity Check . . . 8
1.6 Step 2: BGP Configuration . . . 9
1.7 Step 3: Verifying the Session . . . 9
1.8 Step 4: Inspecting Attributes . . . 10
1.9 Step 5: Breaking the Session . . . 11
1.10 Summary . . . 11

**2 Lab 2 - iBGP, MP-BGP, and Route Origination** 12
2.1 Objectives . . . 12
2.2 Topology & Addressing . . . 12
2.3 Deployment . . . 13
2.4 Task 0: Interface and Loopback Configuration . . . 13
2.5 Task 1: The Internal Foundation (IGP) . . . 13
2.6 Task 2: Establishing iBGP over Loopbacks . . . 14
2.7 Task 3: Originating Prefixes (The Anchor) . . . 15
2.8 Task 4: Solving the Next-Hop Issue . . . 16
2.9 Task 5: Controlled Redistribution . . . 17
2.10 Task 6: Policy Changes Without Resets . . . 18
2.11 Summary . . . 19

**3 Lab 3 - Multi-Homing, Policy, and Traffic Engineering** 20
3.1 Objectives . . . 20
3.2 Topology & Addressing . . . 20
3.3 Deployment . . . 21
3.4 Task 1: Basic Peering, Route Origination and Export Policy . . . 21
3.5 Task 2: Internal Propagation (iBGP) . . . 22
3.6 Task 3: Inbound Filtering (Defensive Hygiene) . . . 23
3.7 Task 4: Traffic Engineering (Local Preference) . . . 24
3.8 Bonus: Failover Test . . . 25

**4 Lab 4 - Capstone: The Regional Service Provider** 26
4.1 Objectives . . . 26
4.2 Topology . . . 26

4.3 Deployment . . . . . 27
4.4 Task 1: The Foundation (IGP & iBGP) . . . . . 27
4.5 Task 2: Customer Onboarding (AS 65100) . . . . . 28
4.6 Task 3: Multi-Provider Transit (TE) . . . . . 28
4.7 Task 4: Advanced Policy (RTBH with BGP Communities) . . . . . 29
4.8 Final Verification Checklist . . . . . 30

# About this course

This course provides a comprehensive introduction to the **Border Gateway Protocol (BGP)**, the "glue" that holds the Internet together. It explores how network operators exchange routing information, enforce policies, and exert control over traffic flows to maintain a stable, loop-free global network.

Unlike theoretical deep dives, this course is designed for **practical observability and hands-on engineering**. We move beyond the "how it works" to the "how to build and fix it." By using modern, containerized environments like **FRR (Free Range Routing)** and **Containerlab**, students can simulate complex, multi-AS topologies on their own machines, gaining the confidence to manage real-world infrastructure.

Key topics covered include:

- **Core Concepts:** Understanding Autonomous Systems (ASNs), the BGP Finite State Machine (FSM), and the fundamental distinction between Internal (iBGP) and External (eBGP) sessions.
- **Protocol Mechanics & Messaging:** A deep dive into BGP message types and the underlying TCP-based transport that ensures reliability.
- **Path Selection & Policy:** Mastering the BGP Best Path Selection algorithm and understanding how Path Attributes (Weight, Local Pref, MED, AS_PATH) are used to manipulate traffic flow.
- **Internal BGP (iBGP):** Navigating the complexities of internal route propagation, the split-horizon rule, and scalable architectures using Route Reflectors.
- **Route Origination & Hygiene:** Professional techniques for announcing prefixes to the world, including the "Exact Match" rule, route anchoring with Null0/blackhole routes, and safe redistribution strategies.
- **Verification & Troubleshooting:** Developing a systematic approach to debugging BGP sessions, including interpreting the BGP table, verifying advertisements, and using soft-resets to apply policy changes without disruption.

## References & Further Reading

### Official Specifications

- **RFC 4271:** A Border Gateway Protocol 4 (BGP-4).
- **RFC 4456:** BGP Route Reflection: An Alternative to Full Mesh Internal BGP.
- **RFC 1997:** BGP Communities Attribute.

### Tools used in this course

- **FRR (Free Range Routing):** The IP routing protocol suite for Linux and Unix platforms.
- **Containerlab:** Orchestration for container-based networking labs.

### Community Resources

- **Hurricane Electric BGP Toolkit:** Essential tool for checking global routing information and ASN details.
- **NANOG BGP Tutorials:** High-quality presentations and videos from the North American Network Operators Group.
- **BGPStream:** A free resource for monitoring BGP hijacking, leaks, and outages.

# 1 Lab 1 - Establishing Basic BGP Connectivity

## 1.1 Objectives

- Configure a basic eBGP session between two routers.
- Verify the BGP Finite State Machine (FSM) transitions.
- Originate and verify a prefix announcement.
- Inspect BGP Path Attributes (AS_PATH, NEXT_HOP).

## 1.2 Working with the Lab Environment

All labs run FRR routers as containers orchestrated by Containerlab, so you need a Linux machine (or VM) with Docker and Containerlab installed. Each lab lives in its own directory under `labs/`:

| Path | Contents |
|---|---|
| `labs/lab-N/lab-N.clab.yml` | The Containerlab topology. |
| `labs/lab-N/configs/` | The starting configuration of each router: hostname and interface addresses, and for routers you don't operate (ISPs, customers), their full configuration. |
| `labs/lab-N/solution/` | The finished configuration of each router. |
| `labs/daemons`, `labs/vtysh.conf` | FRR settings shared by all labs (they enable `bgpd`, `ospfd` and `ospf6d`). |

Useful commands:

```
cd labs/lab-1
sudo containerlab deploy                                  # start the lab
docker exec -it clab-bgp101-lab1-router-a vtysh           # router CLI
docker exec clab-bgp101-lab1-router-a ping -c 3 10.0.0.2  # shell commands
sudo containerlab destroy                                 # tear it down
../load-solution.sh lab-1                                 # apply the solution
```

A few things to keep in mind:

- Use `ping` and `traceroute` from the container shell as shown above. The `ping` command inside `vtysh` doesn't stop on its own.

- The starting configurations are mounted read-only, so `write memory` fails on purpose and your changes are lost when the lab is destroyed. To keep your work, save the running configuration from the host: `docker exec clab-bgp101-lab1-router-a vtysh -c 'show running-config' > router-a.txt`.
- `load-solution.sh` applies the finished configuration on top of the running one. It is meant for instructors, or for comparing your work at the end of a lab.

## 1.3 Topology

We will use a simple point-to-point topology:

- **Router A (AS 65001)**
  - Interface eth1: 10.0.0.1/30
  - Loopback: 10.255.0.1/32
- **Router B (AS 65002)**
  - Interface eth1: 10.0.0.2/30
  - Loopback: 10.255.0.2/32

The interfaces are already addressed when the lab starts.

---

## 1.4 Deployment

Use the provided `lab-1.clab.yml` to start the topology:

```
cd labs/lab-1
sudo containerlab deploy
```

---

## 1.5 Step 1: Connectivity Check

Before BGP can establish a session, the underlying IP connectivity must be functional.

```
$ docker exec clab-bgp101-lab1-router-a ping -c 3 10.0.0.2
PING 10.0.0.2 (10.0.0.2): 56 data bytes
64 bytes from 10.0.0.2: seq=0 ttl=64 time=0.098 ms
64 bytes from 10.0.0.2: seq=1 ttl=64 time=0.121 ms
64 bytes from 10.0.0.2: seq=2 ttl=64 time=0.091 ms
```

## 1.6 Step 2: BGP Configuration

Configure the BGP process and the neighbor relationship. Enter `vtysh` on each router and type `configure terminal` first.

### 1.6.1 Router A

```
router bgp 65001
 bgp router-id 10.255.0.1
 neighbor 10.0.0.2 remote-as 65002
 address-family ipv4 unicast
  network 10.255.0.1/32
 exit-address-family
```

### 1.6.2 Router B

```
router bgp 65002
 bgp router-id 10.255.0.2
 neighbor 10.0.0.1 remote-as 65001
 address-family ipv4 unicast
  network 10.255.0.2/32
 exit-address-family
```

## 1.7 Step 3: Verifying the Session

Observe the BGP state. It should move through `Connect`/`Active` and `OpenSent` to `Established`. Once established, the `State/PfxRcd` column shows the number of prefixes received instead of a state name.

```
router-a# show ip bgp summary
...
Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
10.0.0.2        4      65002         3         3        1    0    0 00:00:07     (Policy) (Policy) N/A
```

The session is up, but no routes are exchanged. **`(Policy)` is FRR's safe default at work**: following RFC 8212, FRR neither accepts nor sends any route over an eBGP session that has no inbound and outbound policy. We will write real policies in Lab 3. For now, turn the check off on both routers:

```
router bgp 65001
 no bgp ebgp-requires-policy
```

The change only affects new updates, so ask for a refresh with a soft reset (Chapter "Announcing My Routes" explains what this does):

```
router-a# clear ip bgp * soft
router-a# show ip bgp summary
...
Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
10.0.0.2        4      65002         7         8        2    0    0 00:00:34            1        2 N/A
```

## 1.8 Step 4: Inspecting Attributes

Check how the prefixes appear in the BGP table and note the attributes discussed in the "Path Attributes and Best Path Selection" chapter.

```
router-a# show ip bgp
BGP table version is 2, local router ID is 10.255.0.1, vrf id 0
Default local pref 100, local AS 65001
Status codes:  s suppressed, d damped, h history, u unsorted, * valid, > best, = multipath,
               i internal, r RIB-failure, S Stale, R Removed
Nexthop codes: @NNN nexthop's vrf id, < announce-nh-self
Origin codes:  i - IGP, e - EGP, ? - incomplete
RPKI validation codes: V valid, I invalid, N Not found

     Network          Next Hop            Metric LocPrf Weight Path
 *>  10.255.0.1/32    0.0.0.0                  0         32768 i
 *>  10.255.0.2/32    10.0.0.2                 0             0 65002 i
```

**Observations:**

- **AS_PATH:** Notice the `Path` attribute for `10.255.0.2/32` shows `65002`.
- **NEXT_HOP:** The `Next Hop` for the remote prefix is the interface IP of Router B. `0.0.0.0` marks the route Router A originated itself.
- **ORIGIN:** The `i` at the end indicates the route was originated inside the AS (the `network` command).
- **Weight:** Locally originated routes get a weight of 32768, so they always win on this router.

Look at a single prefix in detail, and confirm that the route made it into the routing table:

```
router-a# show ip bgp 10.255.0.2/32
BGP routing table entry for 10.255.0.2/32, version 2
Paths: (1 available, best #1, table default)
  Advertised to peers:
  10.0.0.2
  65002
    10.0.0.2 from 10.0.0.2 (10.255.0.2)
      Origin IGP, metric 0, valid, external, best (First path received)

router-a# show ip route bgp
...
B>* 10.255.0.2/32 [20/0] via 10.0.0.2, eth1, weight 1, 00:00:04
```

Finally, test the data plane between the loopbacks:

```
$ docker exec clab-bgp101-lab1-router-a ping -c 3 -I 10.255.0.1 10.255.0.2
```

**Question:** `PfxSnt` in the summary says Router A sends *two* prefixes to Router B, and `show ip bgp neighbors 10.0.0.2 advertised-routes` confirms that it advertises `10.255.0.2/32` back to the router it learned it from. Why doesn't this create a loop? (Hint: look at the AS_PATH Router B receives.)

## 1.9 Step 5: Breaking the Session

Sessions only come up when both ends agree on the parameters. On Router B, replace the neighbor with one that expects the wrong AS:

```
router bgp 65002
 no neighbor 10.0.0.1
 neighbor 10.0.0.1 remote-as 65009
```

Watch the session on Router A. Router B rejects Router A's OPEN with a NOTIFICATION and the session falls back to `Idle`:

```
router-a# show ip bgp neighbors 10.0.0.2
...
  BGP state = Idle
...
  Last reset 00:00:13,  Notification received (OPEN Message Error/Bad Peer AS)
```

Fix the neighbor on Router B (remote AS 65001) and watch the session come back up. Since you deleted the neighbor, you don't need to soft-reset this time.

## 1.10 Summary

In this lab, you established an eBGP session and exchanged routes. You met FRR's eBGP policy requirement, observed the fundamental attributes that BGP uses to track the origin and path of a prefix, and saw how a parameter mismatch keeps a session from being established. The finished configurations are in `labs/lab-1/solution/`.

# 2 Lab 2 - iBGP, MP-BGP, and Route Origination

## 2.1 Objectives

- Establish an iBGP session using Loopback interfaces.
- Originate IPv4 and IPv6 prefixes using the `network` command and "Anchoring".
- Observe and fix the iBGP Next-Hop issue using `next-hop-self`.
- Perform controlled redistribution of connected networks into BGP.
- Apply a policy change and watch it propagate without resetting the session.

## 2.2 Topology & Addressing

We will use a 3-router topology in Containerlab, plus a server on R1's "Lab-Services" LAN. You operate R1 and R2 (AS 65000). R3 is your provider (AS 65001) and is already configured, and so is the server.

```
graph LR
    subgraph AS 65000 [My Network]
        SRV[srv] --- R1[R1 Border]
        R1 --- R2[R2 Internal]
    end
    subgraph AS 65001 [Provider]
        R3[R3]
    end
    R1 --- R3
```

| Device | Interface | IPv4 Address | IPv6 Address | BGP Role |
|---|---|---|---|---|
| **R1** | lo | 10.255.0.1/32 | 2001:db8:ffff::1/128 | AS 65000 Border |
| **R1** | eth1 (to R2) | 10.0.12.1/24 | 2001:db8:12::1/64 | - |
| **R1** | eth2 (to R3) | 10.0.13.1/24 | 2001:db8:13::1/64 | - |
| **R1** | eth3 (LAN) | 172.16.1.1/24 | - | - |
| **R2** | lo | 10.255.0.2/32 | 2001:db8:ffff::2/128 | AS 65000 Internal |
| **R2** | eth1 (to R1) | 10.0.12.2/24 | 2001:db8:12::2/64 | - |
| **R3** | eth1 (to R1) | 10.0.13.3/24 | 2001:db8:13::3/64 | AS 65001 Provider |
| **srv** | eth1 | 172.16.1.10/24 | - | - |

R3 announces `198.51.100.0/24` and `2001:db8:cafe::/48` to you.

---

## 2.3 Deployment

Use the provided `lab-2.clab.yml` to start the topology (see "Working with the Lab Environment" in Lab 1):

```
cd labs/lab-2
sudo containerlab deploy
docker exec -it clab-bgp101-lab2-r1 vtysh
```

---

## 2.4 Task 0: Interface and Loopback Configuration

The interfaces are addressed when the lab starts. In FRR, the loopback is the interface `lo` and is configured like any other interface. Check R1:

```
r1# show running-config
...
interface lo
 ip address 10.255.0.1/32
 ipv6 address 2001:db8:ffff::1/128
exit
r1# show interface brief
```

---

## 2.5 Task 1: The Internal Foundation (IGP)

Before BGP, our internal routers must know how to reach each other's Loopbacks. We will use static routes for simplicity in this lab; a real network would run an IGP such as OSPF or IS-IS. Remember that reachability must work **in both directions**.

### 2.5.1 Configuration (R1)

```
ip route 10.255.0.2/32 10.0.12.2
ipv6 route 2001:db8:ffff::2/128 2001:db8:12::2
```

### 2.5.2 Configuration (R2)

```
ip route 10.255.0.1/32 10.0.12.1
ipv6 route 2001:db8:ffff::1/128 2001:db8:12::1
```

### 2.5.3 Verification

Ensure you can ping R2's loopback from R1's loopback:

```
$ docker exec clab-bgp101-lab2-r1 ping -c 2 -I 10.255.0.1 10.255.0.2
2 packets transmitted, 2 packets received, 0% packet loss
$ docker exec clab-bgp101-lab2-r1 ping -c 2 -I 2001:db8:ffff::1 2001:db8:ffff::2
2 packets transmitted, 2 packets received, 0% packet loss
```

---

## 2.6 Task 2: Establishing iBGP over Loopbacks

Now we establish the BGP session. We must specify the `update-source` so that the TCP session uses the Loopback address. A single session over IPv4 will carry both address families: IPv4 unicast is active by default, and we activate IPv6 unicast explicitly.

### 2.6.1 Configuration (R1)

```
router bgp 65000
 bgp router-id 10.255.0.1
 neighbor 10.255.0.2 remote-as 65000
 neighbor 10.255.0.2 update-source lo
 !
 address-family ipv6 unicast
  neighbor 10.255.0.2 activate
 exit-address-family
```

Configure R2 the same way, with router ID `10.255.0.2` and neighbor `10.255.0.1`.

### 2.6.2 Verification

Check the session state:

```
r1# show bgp summary

IPv4 Unicast Summary:
...
Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
10.255.0.2      4      65000         4         4        0    0    0 00:00:08            0        0 N/A

IPv6 Unicast Summary:
...
Neighbor        V         AS   MsgRcvd   MsgSent   TblVer  InQ OutQ  Up/Down State/PfxRcd   PfxSnt Desc
10.255.0.2      4      65000         4         4        0    0    0 00:00:08            0        0 N/A
```

*Expected Result: The State/PfxRcd column should show a number (e.g., 0), not "Active", "Connect" or "Idle".*

Since this is an iBGP session, FRR's eBGP policy requirement (Lab 1) doesn't apply.

---

## 2.7 Task 3: Originating Prefixes (The Anchor)

We will announce `192.0.2.0/24` and `2001:db8:beef::/48`. Neither exists on any interface, so we anchor them with blackhole routes first.

### 2.7.1 Configuration (R1)

```
ip route 192.0.2.0/24 blackhole
ipv6 route 2001:db8:beef::/48 blackhole
!
router bgp 65000
 address-family ipv4 unicast
  network 192.0.2.0/24
 exit-address-family
 !
 address-family ipv6 unicast
  network 2001:db8:beef::/48
 exit-address-family
```

### 2.7.2 Verification

Check that R2 receives both prefixes:

```
r2# show ip bgp
...
     Network          Next Hop            Metric LocPrf Weight Path
 *>i 192.0.2.0/24     10.255.0.1               0    100      0 i

r2# show bgp ipv6 unicast 2001:db8:beef::/48
BGP routing table entry for 2001:db8:beef::/48, version 1
Paths: (1 available, best #1, table default)
  Not advertised to any peer
  Local
    2001:db8:ffff::1 from 10.255.0.1 (10.255.0.1)
      Origin IGP, metric 0, localpref 100, valid, internal, best (First path received)
```

*Expected Result: The `*>` symbols indicate the route is Valid and Best, and the `i` that it was learned via iBGP. Note that the IPv6 prefix, although carried over an IPv4 session, has an IPv6 next hop: R1's loopback.*

**Try it:** remove the blackhole route for `192.0.2.0/24` and look at R2 again. What happens to the announcement? Put the route back when you're done.

## 2.8 Task 4: Solving the Next-Hop Issue

Now connect R1 to the provider. Configure one eBGP session over IPv4 for IPv4 prefixes and one over IPv6 for IPv6 prefixes. As in Lab 1, we disable the eBGP policy requirement for now.

### 2.8.1 Configuration (R1)

```
router bgp 65000
 no bgp ebgp-requires-policy
 neighbor 10.0.13.3 remote-as 65001
 neighbor 2001:db8:13::3 remote-as 65001
 !
 address-family ipv4 unicast
  no neighbor 2001:db8:13::3 activate
 exit-address-family
 !
 address-family ipv6 unicast
  neighbor 2001:db8:13::3 activate
 exit-address-family
```

### 2.8.2 Observation (on R2)

R3 sends `198.51.100.0/24`. Check the route on the internal router:

```
r2# show ip bgp
...
     Network          Next Hop            Metric LocPrf Weight Path
 *>i 192.0.2.0/24     10.255.0.1               0    100      0 i
   i 198.51.100.0/24  10.0.13.3                0    100      0 65001 i

r2# show ip bgp 198.51.100.0/24
BGP routing table entry for 198.51.100.0/24, version 0
Paths: (1 available, no best path)
  Not advertised to any peer
  65001
    10.0.13.3 (inaccessible, import-check enabled) from 10.255.0.1 (10.255.0.1)
      Origin IGP, metric 0, localpref 100, invalid, internal
```

*Expected Result: Notice the `i` (learned via iBGP) but the lack of `*>`. The Next-Hop `10.0.13.3` is inaccessible to R2! The same happens with `2001:db8:cafe::/48` and its next hop `2001:db8:13::3`.*

### 2.8.3 The Fix (on R1)

```
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.255.0.2 next-hop-self
 exit-address-family
 !
 address-family ipv6 unicast
  neighbor 10.255.0.2 next-hop-self
 exit-address-family
```

### 2.8.4 Verification (on R2)

```
r2# show ip bgp
...
     Network          Next Hop            Metric LocPrf Weight Path
 *>i 192.0.2.0/24     10.255.0.1               0    100      0 i
 *>i 198.51.100.0/24  10.255.0.1               0    100      0 65001 i

r2# show bgp ipv6 unicast
...
 *>i 2001:db8:cafe::/48
                     2001:db8:ffff::1
                                              0    100      0 65001 i
```

*Expected Result: The Next-Hop changed to R1's Loopback, which R2 knows how to reach. The routes are now `*>` (Best).*

**Question:** R2 now has a route to `198.51.100.0/24`, yet `ping -I 10.255.0.2 198.51.100.1` from R2 fails. Why? (Hint: does R3 know how to reach `10.255.0.2`? Should it?)

---

## 2.9 Task 5: Controlled Redistribution

The server on the Lab-Services LAN (`172.16.1.0/24`) can't reach the provider yet: try `docker exec clab-bgp101-lab2-srv ping -c 2 198.51.100.1`. We want to redistribute R1's connected interfaces into BGP, but ONLY the Lab-Services LAN.

### 2.9.1 Configuration (R1)

```
ip prefix-list PL-SERVICES permit 172.16.1.0/24
!
route-map RM-SERVICES permit 10
 match ip address prefix-list PL-SERVICES
!
router bgp 65000
 address-family ipv4 unicast
  redistribute connected route-map RM-SERVICES
```

### 2.9.2 Verification

Ensure other connected networks (the links, the loopback) are NOT in the BGP table:

```
r1# show ip bgp
...
     Network          Next Hop            Metric LocPrf Weight Path
 *>  172.16.1.0/24    0.0.0.0                  0         32768 ?
 *>  192.0.2.0/24     0.0.0.0                  0         32768 i
 *>  198.51.100.0/24  10.0.13.3                0             0 65001 i
```

*Expected Result: Only the permitted subnet appears, with origin* `?` *(incomplete) because it was redistributed. Now the ping from the server to* `198.51.100.1` *works.*

---

## 2.10 Task 6: Policy Changes Without Resets

Modify the `RM-SERVICES` route-map to set a MED of 50, and watch R2.

### 2.10.1 Configuration (R1)

```
route-map RM-SERVICES permit 10
 set metric 50
```

### 2.10.2 Verification (on R2)

```
r2# show ip bgp 172.16.1.0/24
BGP routing table entry for 172.16.1.0/24, version 5
...
      Origin incomplete, metric 50, localpref 100, valid, internal, best (First path received)
```

*Expected Result: After about 5 seconds, the Metric (MED) is 50.*

FRR re-applies a route-map automatically when you edit it (after `bgp route-map delay-timer`, 5 seconds by default) or attach it to a neighbor. Many other implementations, such as Cisco IOS, don't: there you would apply the change with a soft reset, which you can also do in FRR:

```
r1# clear ip bgp 10.255.0.2 soft out
```

The session stays up: check its `Up/Down` timer in `show bgp summary`.

---

## 2.11 Summary

You built an iBGP session over loopbacks that carries both IPv4 and IPv6, originated anchored prefixes, fixed the iBGP next-hop problem, and redistributed a single connected network under the control of a route-map. The finished configurations of R1 and R2 are in `labs/lab-2/solution/`.

# 3 Lab 3 - Multi-Homing, Policy, and Traffic Engineering

## 3.1 Objectives

- Implement BGP in a dual-homed environment with two different ISPs.
- Write real inbound and outbound eBGP policies, and keep your network from becoming a transit path between your ISPs.
- Originate prefixes with high availability using "Anchoring" on multiple border routers.
- Solve iBGP propagation issues in a non-full-mesh environment.
- Implement inbound filtering to keep unwanted routes out of your network.
- Use **Local Preference** to steer outbound traffic toward a primary ISP.

## 3.2 Topology & Addressing

This lab uses five routers: three in your AS (AS 65000) and two representing external ISPs. Inside your AS, OSPF is already running and advertises the loopbacks and the internal links. The ISPs are fully configured.

```
graph LR
    subgraph AS 65001 [ISP-1]
        ISP1[ISP-1]
    end

    subgraph AS 65000 [My Network]
        R1[Border-A] --- R2[Core]
        R2 --- R4[Border-B]
    end

    subgraph AS 65002 [ISP-2]
        ISP2[ISP-2]
    end

    ISP1 --- R1
    R4 --- ISP2
```

| Device | Interface | IP Address | Role |
|---|---|---|---|
| **R1** | lo | 10.255.0.1/32 | Border-A (AS 65000) |
| **R1** | eth1 (to ISP-1) | 10.0.1.1/24 | |
| **R1** | eth2 (to R2) | 10.0.12.1/24 | |
| **R2** | lo | 10.255.0.2/32, 192.0.2.2/32 | Core (AS 65000) |

| Device | Interface | IP Address | Role |
|---|---|---|---|
| **R2** | eth1 (to R1) | 10.0.12.2/24 | |
| **R2** | eth2 (to R4) | 10.0.24.2/24 | |
| **R4** | lo | 10.255.0.4/32 | Border-B (AS 65000) |
| **R4** | eth1 (to R2) | 10.0.24.4/24 | |
| **R4** | eth2 (to ISP-2) | 10.0.2.4/24 | |
| **ISP-1** | eth1 | 10.0.1.100/24 | AS 65001 |
| **ISP-2** | eth1 | 10.0.2.100/24 | AS 65002 |

`192.0.2.2` on R2 plays the role of a server inside your network; we'll use it as the source of test traffic. Both ISPs send you a default route, their own prefix (`198.51.100.0/24` from ISP-1, `203.0.113.0/24` from ISP-2), and `198.18.0.0/24`, a remote network both of them can reach. Each ISP also sends a couple of routes you shouldn't accept.

---

## 3.3 Deployment

Use the provided `lab-3.clab.yml` to start the topology:

```
cd labs/lab-3
sudo containerlab deploy
```

Check that OSPF is up before you start (`show ip ospf neighbor` on R2 should list R1 and R4).

---

## 3.4 Task 1: Basic Peering, Route Origination and Export Policy

Your task is to announce your network `192.0.2.0/24` to the world, from both border routers.

1. **Anchoring:** On both R1 and R4, create a blackhole route for your network.

   ```
   ip route 192.0.2.0/24 blackhole
   ```

2. **Policies:** This time we keep FRR's eBGP policy requirement (Lab 1) and write the policies it asks for. Outbound, announce **only** your own prefix. Inbound, accept everything for now; Task 3 tightens it. On R1:

   ```
   ip prefix-list PL-MY-PREFIXES seq 5 permit 192.0.2.0/24
   !
   route-map RM-ISP1-OUT permit 10
    match ip address prefix-list PL-MY-PREFIXES
   exit
   !
   route-map RM-ISP1-IN permit 10
   exit
   ```

A route-map entry without `match` statements matches every route.

3. **BGP Config:** Enable BGP on R1, establish eBGP with ISP-1 and apply the policies:

```
router bgp 65000
 bgp router-id 10.255.0.1
 neighbor 10.0.1.100 remote-as 65001
 address-family ipv4 unicast
  network 192.0.2.0/24
  neighbor 10.0.1.100 route-map RM-ISP1-IN in
  neighbor 10.0.1.100 route-map RM-ISP1-OUT out
 exit-address-family
```

Do the same on R4 towards ISP-2 (`10.0.2.100`, AS 65002), with route-maps named `RM-ISP2-IN` and `RM-ISP2-OUT`.

4. **Verification:** Check ISP-1 and ISP-2. Do they see `192.0.2.0/24` in their BGP tables?

```
isp1# show ip bgp
...
 *>  192.0.2.0/24     10.0.1.1                 0             0 65000 i
```

---

## 3.5 Task 2: Internal Propagation (iBGP)

Now, establish iBGP sessions over the loopbacks between **R1-R2** and **R4-R2** (OSPF already provides reachability between the loopbacks):

```
! On R1 and R4
router bgp 65000
 neighbor 10.255.0.2 remote-as 65000
 neighbor 10.255.0.2 update-source lo
!
! On R2
router bgp 65000
 bgp router-id 10.255.0.2
 neighbor 10.255.0.1 remote-as 65000
 neighbor 10.255.0.1 update-source lo
 neighbor 10.255.0.4 remote-as 65000
 neighbor 10.255.0.4 update-source lo
```

1. **The Goal:** R2 should learn the routes of both ISPs, from R1 and R4.
2. **Problem 1:** Look at the BGP table on R2 (`show ip bgp`). Why are the routes learned from the ISPs not valid (no `*`)?

```
r2# show ip bgp
...
   i 0.0.0.0/0        10.0.1.100               0    100      0 65001 i
   i                  10.0.2.100               0    100      0 65002 i
```

3. **Fix 1:** Apply `next-hop-self` on R1 and R4 for their iBGP sessions toward R2.

```
! Example on R1
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.255.0.2 next-hop-self
 exit-address-family
```

4. **Problem 2:** R2 now has valid routes from both ISPs. Now look at R4: it only knows ISP-2's routes, and R1 only knows ISP-1's. Why doesn't R2 pass them along? (Hint: the iBGP Split-Horizon rule.)

5. **Fix 2:** Make R2 a **Route Reflector** with R1 and R4 as its clients:

```
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.255.0.1 route-reflector-client
  neighbor 10.255.0.4 route-reflector-client
 exit-address-family
```

6. **Verification:** R4 now learns ISP-1's prefix from R2, with the route reflection attributes:

```
r4# show ip bgp 198.51.100.0/24
BGP routing table entry for 198.51.100.0/24, version 8
Paths: (1 available, best #1, table default)
  Not advertised to any peer
  65001
    10.255.0.1 (metric 20) from 10.255.0.2 (10.255.0.1)
      Origin IGP, metric 0, localpref 100, valid, internal, best (First path received)
      Originator: 10.255.0.1, Cluster list: 10.255.0.2
```

**Question:** R4 now has ISP-1's routes, but "Not advertised to any peer" says it doesn't pass them on to ISP-2. What would happen without the `RM-ISP2-OUT` export policy? Check ISP-2's table: it should only contain `192.0.2.0/24` from you.

---

## 3.6 Task 3: Inbound Filtering (Defensive Hygiene)

The ISPs send you routes that shouldn't be in your table: private address space ("bogons") and prefixes longer than /24, which most networks don't accept. Your own prefix should never come back to you from outside either.

1. **See what you receive:** Enable `soft-reconfiguration inbound` towards the ISP, so that you can see the routes before your policy is applied:

```
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.1.100 soft-reconfiguration inbound
```

2. **Define Policy:** Create a prefix-list that denies bogons and your own prefix, and permits the default route and any prefix from /8 to /24:

```
ip prefix-list PL-ISP-IN seq 5 deny 0.0.0.0/8 le 32
ip prefix-list PL-ISP-IN seq 10 deny 10.0.0.0/8 le 32
ip prefix-list PL-ISP-IN seq 15 deny 100.64.0.0/10 le 32
ip prefix-list PL-ISP-IN seq 20 deny 127.0.0.0/8 le 32
ip prefix-list PL-ISP-IN seq 25 deny 169.254.0.0/16 le 32
ip prefix-list PL-ISP-IN seq 30 deny 172.16.0.0/12 le 32
ip prefix-list PL-ISP-IN seq 35 deny 192.168.0.0/16 le 32
ip prefix-list PL-ISP-IN seq 40 deny 224.0.0.0/3 le 32
ip prefix-list PL-ISP-IN seq 45 deny 192.0.2.0/24 le 32
ip prefix-list PL-ISP-IN seq 100 permit 0.0.0.0/0
ip prefix-list PL-ISP-IN seq 105 permit 0.0.0.0/0 ge 8 le 24
```

A complete bogon list would also deny the documentation and benchmarking ranges, which this course uses as if they were public addresses.

3. **Apply:** Match the prefix-list in the inbound route-map, and add a prefix limit:

```
route-map RM-ISP1-IN permit 10
 match ip address prefix-list PL-ISP-IN
!
router bgp 65000
 neighbor 10.0.1.100 maximum-prefix 100
```

Do the same on R4 for ISP-2.

4. **Verification:** Compare what ISP-1 sends with what you keep:

```
r1# show ip bgp neighbors 10.0.1.100 received-routes
...
 *> 0.0.0.0/0        10.0.1.100               0             0 65001 i
 *> 10.99.0.0/16     10.0.1.100               0             0 65001 i
 *> 192.0.2.0/24     10.0.1.100                             0 65001 65000 i
 *> 198.18.0.0/24    10.0.1.100               0             0 65001 i
 *> 198.51.100.0/24  10.0.1.100               0             0 65001 i
 *> 198.51.100.128/25
                     10.0.1.100               0             0 65001 i

Total number of prefixes 6 (3 filtered)

r1# show ip bgp neighbors 10.0.1.100 routes
...
 *>  0.0.0.0/0        10.0.1.100               0             0 65001 i
 *>  198.18.0.0/24    10.0.1.100               0             0 65001 i
 *>  198.51.100.0/24  10.0.1.100               0             0 65001 i
```

---

## 3.7 Task 4: Traffic Engineering (Local Preference)

Both ISPs can reach `198.18.0.0/24`, and both send you a default route. Look at the paths to `198.18.0.0/24` before changing anything:

- **R4** prefers its own eBGP path via ISP-2 over the iBGP path via R1 ("prefer eBGP over iBGP").

- **R2** has two iBGP paths that tie on every attribute. Which one wins, and which step of the Best Path Selection algorithm decides it?

We want the whole AS to **always** use ISP-1 as the exit point, as long as it is available.

1. **Identify the Tool:** We will use **Local Preference**, which travels with the route to every router in the AS.
2. **Configuration (R1):** Set the local-pref to 200 in the inbound route-map from ISP-1:

   ```
   route-map RM-ISP1-IN permit 10
    set local-preference 200
   ```

   FRR applies the change by itself after a few seconds. On other platforms, you would now run `clear ip bgp 10.0.1.100 soft in`.
3. **Verification:**

   - On R4, run `show ip bgp`. The path via R1 (`10.255.0.1`, LocPrf 200) is now preferred over R4's own eBGP path:

     ```
     r4# show ip bgp
     ...
      *>i 0.0.0.0/0        10.255.0.1               0    200      0 65001 i
      *                    10.0.2.100               0             0 65002 i
      ...
      *>i 198.18.0.0/24    10.255.0.1               0    200      0 65001 i
      *                    10.0.2.100               0             0 65002 i
     ```

   - Run a `traceroute` from R2's server address to the remote network. Does it exit via R1?

     ```
     $ docker exec clab-bgp101-lab3-r2 traceroute -n -s 192.0.2.2 198.18.0.1
     traceroute to 198.18.0.1 (198.18.0.1) from 192.0.2.2, 30 hops max, 46 byte packets
      1  10.0.12.1  0.003 ms  0.001 ms  0.002 ms
      2  198.18.0.1  0.002 ms  0.001 ms  0.002 ms
     ```

**Question:** Why do we source the traceroute from `192.0.2.2` instead of R2's loopback, `10.255.0.2`?

---

## 3.8 Bonus: Failover Test

1. Shut down the interface on R1 toward ISP-1 (`interface eth1`, `shutdown`).
2. Watch how R2 and R4 update their BGP tables to use the path via R4 (ISP-2), and repeat the traceroute: the first hop is now `10.0.24.4`.
3. Bring the interface back up (`no shutdown`) and watch the traffic return to R1 once the Local-Pref 200 routes are re-learned.

The finished configurations of R1, R2 and R4 are in `labs/lab-3/solution/`.

# 4 Lab 4 - Capstone: The Regional Service Provider

## 4.1 Objectives

- Build a scalable ISP core using **OSPF** and **iBGP Route Reflection**.
- Implement secure customer onboarding with prefix filtering and limits.
- Manage multi-provider transit using **Local Preference** and **AS-PATH Prepending**.
- Implement **Remote Triggered Blackholing (RTBH)** for your customers using **BGP Communities**.

## 4.2 Topology

You are operating **AS 65000**. Your network consists of a central core router and two border routers. This time the routers of AS 65000 only have their interfaces addressed: everything else is up to you. The upstreams and the customer are configured.

```
graph TD
    subgraph AS65000 [My ISP - AS 65000]
        C1[CORE-1 / RR] --- BW[BR-WEST]
        C1 --- BE[BR-EAST]
    end

    UP1[UP-1 / AS 65001] --- BW
    UP2[UP-2 / AS 65002] --- BE
    CUST[CUST-A / AS 65100] --- C1
```

| Device | Interface | IP Address | Role |
|---|---|---|---|
| **CORE-1** | lo | 10.255.0.1/32 | Core, RR (AS 65000) |
| **CORE-1** | eth1 (to BR-WEST) | 10.0.12.1/24 | |
| **CORE-1** | eth2 (to BR-EAST) | 10.0.13.1/24 | |
| **CORE-1** | eth3 (to CUST-A) | 10.0.100.1/30 | |
| **BR-WEST** | lo | 10.255.0.2/32 | Border (AS 65000) |
| **BR-WEST** | eth1 (to CORE-1) | 10.0.12.2/24 | |
| **BR-WEST** | eth2 (to UP-1) | 10.0.1.2/24 | |
| **BR-EAST** | lo | 10.255.0.3/32 | Border (AS 65000) |
| **BR-EAST** | eth1 (to CORE-1) | 10.0.13.3/24 | |
| **BR-EAST** | eth2 (to UP-2) | 10.0.2.3/24 | |

| Device | Interface | IP Address | Role |
|---|---|---|---|
| **UP-1** | eth1 | 10.0.1.100/24 | Upstream (AS 65001) |
| **UP-2** | eth1 | 10.0.2.100/24 | Upstream (AS 65002) |
| **CUST-A** | eth1 | 10.0.100.2/30 | Customer (AS 65100) |
| **CUST-A** | lo | 203.0.113.1/32, 203.0.113.50/32 | |

- Your address block is `192.0.2.0/24`.
- Both upstreams send a default route and `198.18.0.0/24`, a remote network with a test address, `198.18.0.1`.
- CUST-A has been assigned `203.0.113.0/24`.

---

## 4.3 Deployment

Use the provided `lab-4.clab.yml` to start the topology:

```
cd labs/lab-4
sudo containerlab deploy
```

---

## 4.4 Task 1: The Foundation (IGP & iBGP)

Before BGP can work, your internal routers must be able to reach each other's Loopbacks.

1. **OSPF:** Enable OSPF area 0 on the internal links and loopbacks of AS 65000. In FRR, you can enable it per interface (`ip ospf area 0`); remember to set a router ID under `router ospf`. Also enable it on CORE-1's link to CUST-A as a **passive** interface: the border routers will need to reach CUST-A's address, which is the next hop of CUST-A's routes.
2. **iBGP Route Reflection:**
   - Configure CORE-1 as a **Route Reflector**.
   - BR-WEST and BR-EAST should be neighbors of CORE-1 (using loopbacks).
   - On CORE-1, set the border routers as `route-reflector-client`.
3. **Verification:** `show ip ospf neighbor` on CORE-1 lists both border routers, and `show bgp summary` shows both iBGP sessions established.

---

## 4.5 Task 2: Customer Onboarding (AS 65100)

CUST-A is a new customer. You have assigned them the block `203.0.113.0/24`. Remember that FRR requires inbound and outbound policies on every eBGP session (Lab 3).

1. **Strict Filtering:** Create a prefix-list on CORE-1 that *only* allows `203.0.113.0/24` from CUST-A, and apply it inbound with a route-map.
2. **Outbound:** Send CUST-A all your routes (a route-map that permits everything).
3. **Prefix Limit:** Implement `maximum-prefix 5` on the neighbor session to prevent them from sending too many routes.
4. **Verification:**
   - On CUST-A, `show ip bgp neighbors 10.0.100.1 advertised-routes` shows what the customer sends. Is there anything that shouldn't be there? Does it make it past your filter (`show ip bgp neighbors 10.0.100.2 routes` on CORE-1)?
   - Ensure CORE-1 sees `203.0.113.0/24` and reflects it to the border routers.

---

## 4.6 Task 3: Multi-Provider Transit (TE)

You have two upstreams. UP-1 (AS 65001) is your primary provider.

1. **Origination and Export:** Announce `192.0.2.0/24` from both border routers (anchored, as in Lab 3). Towards the upstreams, announce **only** your block and your customer's: `192.0.2.0/24` and `203.0.113.0/24`. Use `next-hop-self` towards CORE-1.
2. **Outbound TE (Local-Pref):** On BR-WEST, set a **Local Preference of 200** for all routes received from UP-1.
3. **Inbound TE (AS-Path Prepending):** On BR-EAST, make your network look less attractive to the world. Prepend your ASN three times (`65000 65000 65000`) for all outbound advertisements to UP-2.
4. **Verification:**
   - Check CORE-1: Does it prefer the path through BR-WEST (`10.255.0.2`, LocPrf 200) for external routes?
   - Check UP-2: Does it see your prefixes with a long AS_PATH?

```
up2# show ip bgp
...
 *>  192.0.2.0/24     10.0.2.3                 0             0 65000 65000 65000 65000 i
 *>  203.0.113.0/24   10.0.2.3                               0 65000 65000 65000 65000 65100 i
```

   - From CUST-A, `traceroute -n -s 203.0.113.1 198.18.0.1` should go through CORE-1 and BR-WEST.

---

## 4.7 Task 4: Advanced Policy (RTBH with BGP Communities)

CUST-A wants a way to "Blackhole" specific IPs during a DDoS attack: they announce the attacked address as a /32 tagged with the community `65000:666`, and your network drops all traffic to it at the border, before it reaches the customer's link.

The classic way to implement this (RFC 5635) uses a **discard next hop**: an address that every router in your AS routes to a blackhole. Routes whose next hop is the discard address are blackholed wherever they are installed.

1. **Discard Route:** On CORE-1, BR-WEST and BR-EAST, route the discard address `192.0.2.1` to a blackhole:

```
ip route 192.0.2.1/32 blackhole
```

2. **Define the Policy (CORE-1):** Add an entry in front of CUST-A's inbound route-map. It accepts /32s inside the customer's block **only** when they carry `65000:666`, sets their next hop to the discard address, and adds `no-export` so they never leave your AS:

```
ip prefix-list PL-CUST-A-RTBH seq 5 permit 203.0.113.0/24 ge 32
bgp community-list standard CL-RTBH seq 5 permit 65000:666
!
route-map RM-CUST-A-IN permit 10
 match ip address prefix-list PL-CUST-A-RTBH
 match community CL-RTBH
 set ip next-hop 192.0.2.1
 set community no-export additive
exit
```

   Your strict filter from Task 2 becomes the next entry of the same route-map (e.g., `permit 20`).

3. **Trigger (CUST-A):** First check that the server answers: `docker exec clab-bgp101-lab4-up1 ping -c 2 -I 198.18.0.1 203.0.113.50`. Then, on CUST-A, announce `203.0.113.50/32` with the community `65000:666`:

```
ip prefix-list PL-RTBH seq 5 permit 203.0.113.50/32
!
route-map RM-TO-AS65000 permit 10
 match ip address prefix-list PL-RTBH
 set community 65000:666
exit
!
route-map RM-TO-AS65000 permit 20
exit
!
router bgp 65100
 address-family ipv4 unicast
  network 203.0.113.50/32
  neighbor 10.0.100.1 route-map RM-TO-AS65000 out
```

4. **Verification:** Check the routing table on BR-WEST. Traffic for `203.0.113.50` is now discarded:

```
br-west# show ip route 203.0.113.50
Routing entry for 203.0.113.50/32
  Known via "bgp", distance 200, metric 0, best
  ...
    192.0.2.1 (recursive), weight 1
  *   unreachable (blackhole), weight 1
```

The ping from UP-1 to `203.0.113.50` now fails, while `203.0.113.1` still answers. Check that neither upstream received the /32.

---

## 4.8 Final Verification Checklist

- ☐ CORE-1 has iBGP sessions up with both Border routers.
- ☐ OSPF is providing reachability between loopbacks.
- ☐ CORE-1 only accepts CUST-A's assigned block (and tagged /32s inside it).
- ☐ CUST-A can reach the simulated internet (`198.18.0.1`).
- ☐ Outbound traffic from the ISP prefers UP-1.
- ☐ UP-2 sees your prefixes with a prepended AS_PATH, and neither upstream sees anything but `192.0.2.0/24` and `203.0.113.0/24`.
- ☐ The Blackhole community correctly drops traffic for the targeted IP at the border.

The finished configurations are in `labs/lab-4/solution/`, including the change on CUST-A.